

Session 4 speaking script: Ranges and Algorithms

How to use this script

- **Bold lines** are the must-say sentences. If you say nothing else on a slide, say those.
- Plain paragraphs are the talk track, written to be read aloud as is. Paraphrase freely once you know them.
- (pause) and (beat) are deliberate stops. A pause is two full seconds of silence; a beat is one. Do not skip them; on Teams, silence is when people unmute.
- >> DO: lines are actions (terminal, editor, Compiler Explorer). >> ASK: lines are questions to the room, with the answer you expect and what to say next.
- >> IF AHEAD: is 60 to 120 seconds of real extra depth for when you are early. >> IF BEHIND: is the one sentence that replaces the slide's talk track.
- Slide numbers count the title slide as 1. The deck has 49 slides.

Pace plan for a fast speaker

Checkpoint (slide)	Target clock
6. Range concepts (Foundation starts)	0:10
17. Foundation takeaway	0:33 (leave at 0:35)
18. Views are lazy (Views starts)	0:35
29. Views takeaway	0:59 (leave at 1:00)
30. What C++20 was missing (C++23 starts)	1:00
37. Execution policies (Parallel starts)	1:20
40. Parallel takeaway	1:29 (leave at 1:30)
41. C++17 numerics (Algorithms starts)	1:30
46. Guidance for production code (Exercise segment starts)	1:40
48. Exercise launched, people working	1:46
49. Session takeaway	1:58

If you hit a checkpoint more than 3 minutes early, use the IF AHEAD material in the next segment rather than speeding on. One exception: from the 1:30 checkpoint onward, bank any early time for the exercise instead. The last segment is overbooked (a 20-minute exercise plus three other slides in 20 minutes; see Deck issues), so every minute you save after 1:30 is a minute of hands-on time.

Slow down deliberately on these four:

- **Slide 14, Borrowed ranges and dangling.** It is the first time the library changes a return *type* based on value category. People nod and do not get it. Say it twice.

- **Slide 18, Views are lazy.** The "6, not 4" result is subtle and is the mental model for everything after it. If they do not get lookahead here, slide 25 makes no sense.
- **Slide 23, The const view trap.** Counterintuitive for C++ veterans who were taught "take by const&". It is the at-home task 4 and it will be a code-review argument back at work.
- **Slide 24, Dangling with views.** It compiles, it runs, it is UB. This is the one that ships to the field. Give it room.

Before class checklist

- Build once on the compiler you present with. GCC 14 is the better presenting toolchain today: it has every C++23 view in the deck (libc++ 18 prints "not available" for windows, the zip family, and the folds).
 - `cmake -S . -B build -G Ninja && cmake --build build`
 - `ctest --test-dir build -R s04 --output-on-failure`
 - Binaries are `./build/demos/s04/demo_s04_<stem>`, for example `./build/demos/s04/demo_s04_lazy`.
- Check TBB: `ldd build/demos/s04/demo_s04_parallel | grep tbb`. If nothing prints, the par numbers will be sequential. Know which you have before slide 37.
- Demo files to open on the right, in order: `range_concepts.cpp`, `constrained_algorithms.cpp`, `projections.cpp`, `algorithm_results.cpp`, `sentinels.cpp`, `dangling.cpp`, `lazy.cpp`, `adaptors_tour.cpp`, `const_view_trap.cpp`, `cost_model.cpp`, `adaptor_closure.cpp`, `ranges_to.cpp`, `zip_enumerate.cpp`, `chunk_by.cpp`, `windows.cpp`, `zip_family.cpp`, `fold.cpp`, `cpp23_algorithms.cpp`, `parallel.cpp`, `numeric17.cpp`, `same_task_three_ways.cpp`. Pin them as tabs in that order so Ctrl+Tab walks forward.
- Compiler Explorer tabs to preload (x86-64 GCC 14, `-std=c++23 -O2`). The demo headers still say `<add short link>`, so paste the files in by hand:
 1. `cost_model.cpp`, with a second compiler pane on Clang 18 `-stdlib=libc++`.
 2. `const_view_trap.cpp` with `-DSHOW_ERRORS` already in the flags (or be ready to type it).
 3. `same_task_three_ways.cpp`, assembly pane open, "Filter: library functions" on.
- Exercise: confirm `exercises/s04-ranges/starter/` and `solution/` are present in your checkout (the staged copy has only the README). On the starter, do task 3 yourself once and confirm the `split("", ' ', '')` test fails exactly as the slide promises, and that `s04_report_identical` passes on the untouched starter.
- `handouts/cheat-sheet-ranges.md` is a stub (one line). Either finish it or do not point at it on slide 49.
- Have the exercise README open in a background tab for the launch.

The script

1. The Evolution of C++: Session 4, Ranges and Algorithms • target 0:00, ~1 min

Welcome back. Session 4. **Today is the biggest change to how we write loops since range-based for.** (beat)

Here's the shape of it. C++20 gave us the model: concepts for ranges, algorithms that take a whole container, projections, and lazy views you compose with a pipe. C++23 filled in the pieces that make it something you'd actually use every day.

One warning up front. If you're on Clang with libc++ 18, you're going to see several of today's C++23 views print "not available". That's not you, that's the library. The support matrix near the end lists every gap, and the exercise avoids all of them. (pause)

Let's look at where the two hours go.

2. Agenda • target 0:01, ~1 min

Seven blocks. Ten minutes of recap and framing. Then the C++20 foundation for twenty-five minutes: concepts, constrained algorithms, projections. Then views and pipelines for twenty-five. Then twenty minutes on what C++23 added. Then two short blocks: parallel algorithms, and the smaller algorithm additions from 17 through 23. Then the exercise.

The exercise starter is the Session 3 solution, same telemetry program, same tests. So everything you see on a slide today with the words "from the exercise" is code you've already touched. (beat)

Quick look back at Session 3 first.

3. Session 3 recap • target 0:02, ~3 min

The Session 3 solution: a constexpr CRC table and crc16, a consteval validator that checks the sensor table at compile time, four constrained serialize templates behind a Serializable concept, and deducing this on SensorStats::add.

Three places people got stuck. First: std::as_bytes in a constexpr function. It's a reinterpret_cast underneath, and a reinterpret_cast is never a constant expression. No compiler flag fixes that. (beat)

Second: transform(&SensorConfig::units) on a temporary optional. Still. This is the second session in a row. The member pointer yields a reference into an rvalue, and the monadic transform won't build an optional of a reference. Use a lambda that returns by value.

Third, and this is the one that matters for today. SerializableRange without the ! StringLike exclusion. A std::string matched both the string overload and the range overload, and the call was ambiguous.

ASK: "Why was a std::string a match for the range overload at all?" Expected: because a string has begin() and end(), so it is a range. Say: "Right. **A string is a range of characters, and that fact comes back today with views::split.**"

(pause)

Keep that in your head. Now, what's wrong with the code we've been writing for twenty years?

IF BEHIND: "Session 3's three stumbles: `as_bytes` is a `reinterpret_cast`, member-pointer transform on a temporary optional, and a string is a range. That last one matters today."

4. The iterator-pair problem • target 0:05, ~3 min

These four lines are in the Session 3 solution today, in `stats.cpp`. Look at them as a reviewer would.

Line 1, the `stable_sort`: `matching.begin()`, `matching.end()`, and a comparator lambda whose whole job is to say "compare by value, descending". Line 2, the `lower_bound`: same pair, and a lambda that says "compare by `ts`". Line 3, the `copy_if` into a `back_inserter`: a pair, an output iterator, and a lambda that says "the sensor matches". Line 4 isn't even an algorithm. It's a hand-written loop tracking a running min and max.

ASK: "Count the lambdas. How many, and what do they say?" Expected: three. Two say "compare by this member", one is a filter predicate. Say: "Two of the three exist only to name a member. That's the part ranges deletes outright."

(pause)

Three problems. **Every one of these takes two iterators that must match, and nothing checks that they do.** A classic review finding is `std::sort(a.begin(), b.end())` after a copy-paste. It compiles. It's undefined behavior. It often appears to work in a unit test with small inputs.

Second, the comparator lambdas. Each one is four lines of ceremony around one member name, and every one is a chance to write `<` where you meant `>`, or `<=`, which isn't a strict weak ordering and corrupts the sort.

Third, no composition. Filter, then sort, then truncate means a temporary vector between each step, because the algorithms can't hand their output to each other.

In a long-lived codebase, this is the code people copy. So it multiplies. (beat)

Here's the fix in one sentence.

IF AHEAD: The `<=` comparator is worth thirty seconds. `std::sort` requires a strict weak ordering; `a <= a` is true, which violates irreflexivity. `libstdc++`'s `introsort` can then walk off the end of the buffer in its unguarded insertion sort when many elements compare equal. Building with `-D_GLIBCXX_DEBUG` adds irreflexivity checks to the sorting algorithms and catches some of these at run time; it's worth turning on in a CI test configuration. Projections reduce the risk because the default comparator is `ranges::less`, which you didn't write.

5. Ranges in one sentence • target 0:08, ~2 min

Ranges are the standard algorithms with the iterator pairs removed, plus lazy views that compose with a pipe. (pause)

Two lines on the slide. The first line: `std::ranges::sort(records, {}, &Record::value)`. The container goes in whole. The empty braces are the comparator, defaulted to "less". And `&Record::value` is a projection: "sort these by their value". No lambda.

The second line: `records | filter(is_rpm) | take(3)`. That builds a view. A recipe. Nothing runs on that line. It runs when you iterate it, and it stops as soon as it has three.

So set expectations. The C++20 half, the algorithms and projections, is what you can use tomorrow morning in any code built as C++20. It's strictly better than what you have. The C++23 half, to, zip, enumerate, chunk_by, fold_left, is what makes views worth using in production, because before 23 you couldn't even turn a view back into a vector cleanly.

That's the arc of today: foundation, views, the C++23 completion, then parallel and the smaller algorithms. (beat)

Let's start with what a range actually is.

IF BEHIND: "Ranges: algorithms that take the container, projections instead of comparators, and lazy pipelines. C++20 is the model, C++23 makes it usable."

6. Range concepts • C++20 • target 0:10, ~3 min

A range is anything with `begin()` and `end()`. The concepts tell you what kind.

This whole slide is `static_asserts`. It's `demos/s04/range_concepts.cpp`, and if it compiles, every line is true.

There's a ladder. `input_range` at the bottom: you can walk it once. `forward_range`: you can walk it more than once. `bidirectional_range`: you can go backward.

`random_access_range`: you can jump, `it + n`, in constant time. `contiguous_range`: the elements sit in memory one after another, so `data()` gives you a pointer.

Each rung includes the ones below it. Line 6 on the slide says it: a vector is also an input range, a forward range, a bidirectional range.

Now walk the first block. Line 3: vector, span, and `string_view` are contiguous. Line 4: deque is random access but not contiguous; it's chunks. Line 5: `list` is bidirectional, not random access. Line 6: `forward_list` only goes forward.

Now the line to look at. Line 11 and 12, the `filter` lines. **Put a vector through `views::filter` and it drops from contiguous all the way down to bidirectional.** (pause) Why? Because to jump ahead five matches, you have to test the predicate on every element in between. There's no `O(1) it + 5` anymore. The concepts carry that information through the pipeline, so an algorithm that needs random access will refuse a filtered range at compile time.

Then two concepts that sit off to the side of the ladder. `sized_range`: `size()` in $O(1)$. `forward_list` isn't. And `view`: cheap to copy, doesn't own. `Span` and `string_view` are views; `vector` isn't. And `borrowed_range`, last line: iterators that can outlive the range object. `Span` yes, `vector` no. Hold that one; it comes back in four slides.

DO: Run `./build/demos/s04/demo_s04_range_concepts`. Expected output: all range concept checks passed at compile time. Point out that the run is a formality: the proof happened in the compiler. Recovery: if it fails to build, read the failing `static_assert` aloud; it is the lesson.

Why does any of this matter to you? Because these concepts are what turn a 200-line template error into one line.

IF AHEAD: A fun check for the room: `std::views::iota(0, 10)` is random access, line 10, even though it doesn't exist in memory. Random access is about iterator arithmetic, not storage. And there's a seventh property not on this slide: `common_range`, where `begin()` and `end()` have the same type. Many views aren't common, because their end is a sentinel. That breaks old code that does `std::vector<int>(v.begin(), v.end())` on a view. Slide 27 is about exactly that.

IF BEHIND: "Input, forward, bidirectional, random access, contiguous; filter drops you to bidirectional; and `view` and `borrowed_range` are the two that matter for lifetimes."

7. Constrained algorithms • C++20 • target 0:13, ~2 min

Left side is what we have: `std::sort`, two iterators, a comparator lambda that says "by value". Right side: `std::ranges::sort(v, {}, &Record::value)`. The braces mean "default comparator", which is `std::ranges::less`. **This is exercise task 1, and it's the change you can make in any C++20 file tomorrow.**

Why a whole new namespace instead of new overloads of `std::sort`? Look at the comment block at the bottom. Five reasons, and I'll pick the three that matter.

One. It takes a range, or an iterator and a sentinel. You can't pass `a.begin()` and `b.end()` by accident when you only pass `a`.

Two. It's constrained. Call `std::ranges::sort` on a `std::list` and GCC says "no match for call to `ranges::__sort_fn`" and then "constraints not satisfied", pointing at `random_access_range`. Call plain `std::sort` on a list and you get a page of errors from deep inside the implementation about `operator-` on list iterators. (beat)

Three. It returns more. Next slide.

And notice where the braces go. The projection is the third argument. **You must write the `{}` to get to the projection position.** A common first mistake is `std::ranges::sort(v, &Record::value)`: that passes the member pointer as the comparator, and you get "no match for call". When you see that error in review, it's almost always the missing braces.

IF AHEAD: Why couldn't the committee just add these to `std::sort`? Because existing code calls `std::sort(first, last)` and expects unconstrained templates, and ADL-found `sort` overloads in user namespaces already exist in the wild. Adding constrained overloads to `std::` would have changed overload resolution in existing code. A parallel namespace was the compatible choice. That's also why both live in `<algorithm>` and you can mix them in one file.

IF BEHIND: "`std::ranges::sort(v, {}, &Record::value)`: the range, a default comparator, a projection. Concept errors instead of template spew."

Projections are the feature I want you to leave with, so let's look at them properly.

8. Projections • C++20 • target 0:15, ~3 min

A projection is applied to each element before the algorithm looks at it. (pause) That's the whole definition. The algorithm compares projected values but hands you back the original elements.

Walk the function on the right with me, `projections.cpp`, line by line.

Line 3: `minmax(v, {}, &Record::value)`. Min and max by value. But look what comes back: `lo` and `hi` are **records**, whole `Record` objects, not doubles. That's why the print line says `lo.value`. People expect a double and get a struct.

Line 4: `lower_bound(v, ts, {}, &Record::ts)`. The old version needed a heterogeneous comparator `lambda`, `Record` on one side and a timestamp on the other. Now you search for a timestamp in a range of records, and the projection bridges the types. Same precondition as always: the range must be sorted by `ts`.

Line 5: `find(kSensors, name, &SensorConfig::name)`. Search the config table by name; get back an iterator to the whole config. That's `find_sensor` in the exercise, in one call.

Line 6: count by sensor. Line 7: sort descending, with `ranges::greater{}` in the comparator slot and the projection after it. Line 8: the projection doesn't have to be a member pointer. Any invocable works. Here it's a lambda returning the sensor name's length, so `max` finds the record with the longest name.

DO: Run `./build/demos/s04/demo_s04_projections`. Expected: 1 3 1 125 2 temperature. Point at each: min value 1, max value 3, the `lower_bound` landed at index 1, the temp config's max is 125, two rpm records, and the longest sensor name is temperature. Recovery: if the build is missing, read the expected output off this script and keep moving.

Every comparator lambda in the starter that just named a member disappears in the solution. That's the exercise's task 1.

ASK: "If `minmax` hands back records, what does it cost when `Record` holds a `std::string`?" Expected: it copies two records, strings and all. Say: "Right, `ranges::minmax` returns by value. If that matters, use `ranges::minmax_element`, which takes the same projection and gives you iterators."

IF AHEAD: Projections compose with the comparator: the comparator receives projected values. So `sort(v, std::ranges::greater{}, &Record::value)` compares doubles with `greater`. A code-review heuristic: if a lambda passed to an algorithm has the shape `[](const T& a, const T& b) { return a.m < b.m; }`, it's a projection waiting to happen. clang-tidy 19's `modernize-use-ranges` will even do the iterator-pair half of the rewrite for you. One edge: a projection to a member of a type with no `operator<`, like a struct, still needs a comparator for that type. The projection doesn't invent ordering.

Next: what these algorithms give back.

9. What the algorithms return • C++20 • target 0:18, ~2 min

The classic algorithms threw information away. `std::copy` gave you the output iterator but not where the input stopped. `std::for_each` gave you the functor but not the end. The `ranges::` versions return structs designed for structured bindings.

Line 1: `minmax` gives the elements. Line 2: `sort` returns the end iterator. Rarely useful, but it's there. Line 3: `copy` returns an `in_out_result`, where both the input and the output stopped. Line 4, the interesting one: `for_each` with a stateful lambda, a mutable `s` that accumulates. You get back the end iterator **and the functor, with its state**.

DO: Run `./build/demos/s04/demo_s04_algorithm_results`. Expected: 1 5 5 14. Min 1, max 5, the copy wrote 5 elements, and `fn(0)` returns 14: the sum of 1, 1, 3, 4, 5 that the lambda accumulated inside `for_each`. Recovery: say the numbers from here.

Now line 6. `std::ranges::find(make(), 2).make()` returns a temporary vector. When `find` returns, that vector is gone. So what does `find` return? Not an iterator. **It returns `std::ranges::dangling`, a type with no operator*.** (pause) Uncomment the `*d` and it doesn't compile. The algorithm refuses to hand you an iterator into an object that no longer exists.

That's a compile-time fix for a whole category of use-after-free. We'll spend a full slide on it shortly.

IF BEHIND: "The ranges algorithms return more: both ends from `copy`, the functor from `for_each`, and `dangling` instead of an iterator into a temporary."

10. Sentinels • C++20 • target 0:20, ~3 min

The end of a range no longer has to be an iterator. It can be a condition. (beat)

Old rule: `begin` and `end` are the same type, and you walk until they're equal. New rule: the end can be a different type, called a sentinel, and all it needs is an `==` against the iterator.

Look at the top of `sentinels.cpp`. `NulSentinel` is an empty struct with one friend `operator==` that takes a `const char*` and returns `*p == '\0'`. That's it. Line 8: `std::ranges::find(s, NulSentinel{}, ',')`. Find a comma in a C string without

calling `strlen` first. One pass instead of two. For anyone who's parsed NUL-terminated buffers off a serial line or out of a legacy API, that's the pattern you write by hand today.

Second use. Line 12: `views::iota(1)` with no upper bound. An infinite range of integers. Transform to squares, then `take_while` less than 50. That's fine, because nothing is computed until you iterate, and the end is a predicate that eventually matches.

Third. Line 17: `std::unreachable_sentinel`. That's a sentinel that never compares equal. You're telling the algorithm "there is a match, trust me, skip the bounds check". The loop drops the end comparison. **If you're wrong, you read off the end of the buffer.** (pause) So it's a tool for a hot loop where you've put a guard value at the end yourself, not a default.

DO: Run `./build/demos/s04/demo_s04_sentinels`. Expected three lines: 5, then 1 4 9 16 25 36 49, then 7. The comma is at index 5, the squares stop before 64, and the w is at index 7. Recovery: read them out.

ASK: "Would you allow `unreachable_sentinel` through code review?" Expected: mixed, probably "only with a comment proving the match exists". Say: "That's the right answer. It's the same review standard as any unchecked index: show me the invariant."

IF AHEAD: The `NulSentinel` works with only `operator==` defined because C++20 synthesizes the reversed `==` and the `!=` from it. That's the Session 1 spaceship material doing quiet work here. And there's a standard spelling for counted ranges too: `std::views::counted(p, n)` gives a span or a subrange of `n` elements from a pointer, which is the safe way to adapt a pointer-and-length C API.

IF BEHIND: "A sentinel is an end that's a condition: NUL-terminated strings without `strlen`, infinite `iota`, and `unreachable_sentinel` when you can prove the match."

11. Customization point objects · C++20 · target 0:23, ~1 min

Quick one. `std::ranges::begin`, `end`, `size`, and every `ranges::` algorithm are objects, not functions. Two consequences.

First, ADL can't hijack them. With `using namespace std::ranges`, an unqualified `sort(v)` finds the object, and a `sort` lurking in `v`'s namespace doesn't get a vote.

Second, **you can pass `std::ranges::sort` to another function as a value, no wrapper `lambda`.** (beat)

One correction to the slide's second bullet: `ranges::begin` tries arrays first, then a member `begin()`, then a free `begin()` found by ADL. And it refuses an rvalue that isn't a borrowed range. That refusal is where dangling comes from.

IF AHEAD: Formally, C++20 only required the algorithms to behave this way (not found by ADL, and suppress ADL when found); every implementation made them objects. C++26 (P3136) makes "they are objects" official, so passing them as values is guaranteed rather than just universal practice.

IF BEHIND: Skip it entirely (the outline lists it on the cut list). Say: "The algorithms are objects, so ADL can't hijack them and you can pass them around."

12. Range-for and ranges • C++20 • target 0:24, ~2 min

Every view is a range, so it works in range-for. Line 1 is the everyday case: `records | filter(is_rpm)` where `records` is a named variable. Fine.

Now line 2, and I need to be precise here, because the slide is subtler than it looks. **The danger in range-for is a temporary that dies before the loop body runs, while the view still points into it.** (pause)

As written, `load().records | take(3)` where `records` is a plain data member is actually safe in C++20. `load().records` is an rvalue vector, and piping an rvalue container moves it into the view, into something called `owning_view`. The view owns the data, and the view lives for the whole loop. Put a print in the destructor and you'll see the struct die before the loop starts, with the records still fine.

The version that bites is an accessor. `load().records()` returning a `const&`. Now the pipe sees an lvalue, wraps it in a `ref_view`, which is just a pointer, and the temporary from `load()` dies at the end of the range expression. **Before C++23, that loop iterates a dead vector.** Try it with a destructor print and the loop prints garbage.

The C++20 fix is line 3: the init-statement from Session 1. `for (auto loaded = load(); ...)`. Name the thing. C++23's P2718 extends every temporary in the range expression to the end of the loop, so the bad line becomes legal. That's GCC 15 and Clang 19. We're on 14 and 18, so you write the init-statement.

Last line: iterate views with `auto&&`. `transform` yields values, `filter` yields references. `auto&&` binds either without a copy.

IF AHEAD: Two review heuristics. One: in a range-for over a pipeline, if the leftmost thing is a function call followed by `.something()`, check whether `something()` returns a reference; that's the dangling shape. Two: `-Wdangling` in Clang and `-Wdangling-reference` in GCC 13+ catch some of the simpler accessor cases on plain range-for, but not through a view pipeline. Don't count on the warning.

IF BEHIND: "Views work in range-for. If the range expression goes through an accessor on a temporary, use the init-statement; C++23 fixes it, our compilers don't yet."

13. `std::ranges::` or `std::?` • target 0:26, ~1 min

The decision table. **Use `std::ranges::` for new code.** Concepts, projections, sentinels, better return values.

Use `std::` in three cases. One, you need an execution policy, `par`: there's no `ranges::` parallel version until C++26. Two, the `<numeric>` family: `accumulate`, `reduce`, `inclusive_scan` have no `ranges::` version. C++23's `fold_left` fills the `accumulate` gap. Three, legacy code that hands you two iterators. And even there,

`std::ranges::subrange(first, last)` turns the pair into a range so you can still use the new algorithms.

Same headers. Mixing them in one file is fine. (beat)

Now the slow slide.

IF BEHIND: "New code: `ranges::`. Old style only for `par`, `<numeric>`, and legacy pairs, and `subrange` bridges those."

14. Borrowed ranges and dangling · C++20 · target 0:27, ~3 min

Slow down here. This is the first time you'll see the library change a return type based on whether you passed it a temporary.

An iterator into a temporary is a bug, and the ranges library refuses to hand you one.
(pause)

Three lines in `dangling.cpp`. Line 2: `find(v, 2)` where `v` is a named vector. You get a normal iterator. Safe, because `v` outlives the call.

Line 3: `find(std::span{v}, 2)`. The span is a temporary. It dies at the semicolon. But you still get a real iterator. Why? Because a span doesn't own anything. Its iterators point into `v`, not into the span. When the span dies, the iterators are still good. That property has a name: **borrowed range**. A range whose iterators can outlive the range object.

Line 4: `find(make(), 2)`. `make()` returns a temporary vector. The vector owns its elements. When it dies, the elements die. So any iterator into it would dangle. The algorithm sees that: `rvalue`, not `borrowed`. And instead of an iterator it returns `std::ranges::dangling`. The `static_assert` on line 5 proves it: the type of `c` is exactly `dangling`.

(pause)

Let me say that again, because it's the idea. **Same function, same arguments, different return type, depending on whether the thing you passed will survive the call.**

Which types are borrowed? Any lvalue, because you named it, so you're responsible for it. `span`, `string_view`, `subrange`, `iota`. Types opt in by specializing `enable_borrowed_range`. Everything else, passed as a temporary, gives you `dangling`.

DO: Point at line 5 of the snippet in `dangling.cpp` on the right. No run needed; the `static_assert` is the demo. If someone asks to see the error, add `*c`; and build: GCC reports no match for `operator*` on `std::ranges::dangling`.

ASK: "`std::ranges::find(std::string("abc"), 'b')`: iterator or `dangling`? And with `std::string_view("abc")`?" Expected: `dangling` for the string (owns its characters), a real iterator for the `string_view` (borrowed; it points at the literal, which lives forever). Say: "Exactly. And the exercise's task 8 is this, deliberately."

This is exercise task 8 at home. It's also the rule that views make easy to break, which is slide 24.

IF AHEAD: If you write your own non-owning range type, a buffer view over DMA memory, say, you opt in with `template<> inline constexpr bool std::ranges::enable_borrowed_range<MyView> = true;`. Only do it if your iterators truly don't point into the object. Getting it wrong silently turns off the dangling protection for your type. That's a good thing to put on a review checklist for anyone writing range types.

15. The concepts, with one type each • target 0:30, ~1 min

This is the handout slide. I won't read the table. Screenshot it.

The one row to say out loud: `views::filter` over a vector is bidirectional, not random access, and it isn't sized. (beat) If you filter, you lose `size()` and you lose `it + n`. Anything downstream that needs those won't compile, which is what you want.

One precision on the `sized_range` row: "all of the above except `forward_list` and `filter`" is a little generous. The `istream` view and `split` results aren't sized either. Neither is anything that can't know its length without walking.

IF BEHIND: "Screenshot it. Filter costs you random access and `size()`."

16. Constrained algorithms with new behavior • target 0:31, ~2 min

Second handout table: the algorithms where the `ranges::` version does something genuinely new.

Rows one and two you've seen: `for_each` gives back the functor, `copy` and `transform` give back both ends. Row three: `minmax`, `min`, `max` return the elements, and the projection is used for comparing, not applied to what you get back. Row four: dangling on temporaries.

Row five deserves a sentence. `ranges::remove` and `ranges::unique` return a subrange, the tail you should erase. So the erase-remove idiom becomes `v.erase(ret.begin(), ret.end())`. Or, simpler, use `std::erase_if(v, pred)` from Session 2.

The rest are C++23: `contains`, `starts_with`, `ends_with`, `find_last`, `fold_left`, `iota`, `shift_left`. **Those rows are uneven on libc++ 18, so check the support matrix on slide 45 before you rely on one.** We'll see most of them in the C++23 block.

IF AHEAD: `ranges::find_last` returning a subrange rather than an iterator is deliberate: when nothing matches, the subrange is empty and starts at end, so "found nothing" and "found it at the end" are distinguishable without a second comparison. `std::find_end` on reverse iterators was the old way, and people got the base-iterator off-by-one wrong constantly.

IF BEHIND: "Second handout table. Note `remove` and `unique` return the subrange to erase."

17. Foundation takeaway • target 0:33, ~2 min

Constrained algorithms and projections are strictly better than iterator pairs. Adopt them in every new line. `std::ranges::sort(v, {}, &T::member)`.

There's no performance cost; they compile to the same code. There's no new lifetime risk; if anything there's less, because of dangling. The only requirement is C++20.

And read dangling as a feature. When you hit it, the library just caught a use-after-free at compile time.

Monday morning: find one `std::sort` with a comparator lambda and replace it with a projection. (pause)

ASK: "Anyone have a codebase where you can't turn on C++20 yet?" Expected: some hands, citing a certified or vendor toolchain. Say: "Then this is your argument for the upgrade request: a whole category of comparator and iterator-pair bugs goes away, with no runtime cost. Put it in the roadmap worksheet at the end of Session 5."

That's the foundation. It's 0:35. Now the part that's actually new: views.

18. Views are lazy · C++20 · target 0:35, ~3 min

Slow down. This is the mental model for the rest of the segment.

A view is a recipe over another range. Nothing runs until you iterate, and iteration stops as early as it can. (pause)

`lazy.cpp` on the right. There's a counter, `calls`, and the filter predicate increments it every time it runs. Ten elements, one through ten. The pipeline is: filter evens, square them, take two.

Line 7 prints the counter right after building the pipeline. Before you see the output, what do you expect?

ASK: "After construction, how many predicate calls? And after iterating, how many?" Expected: zero after construction; most people say four, some say ten, after iteration. Say: "Let's see."

DO: Run `./build/demos/s04/demo_s04_lazy`. Expected output: after construction: 0 predicate calls 4 16 after iteration: 6 predicate calls Point at the 0 first, then the 6. Recovery: if it won't run, read the three lines from here; the explanation is the point.

Zero after construction. Nothing ran. Building a view is just storing a reference and a lambda.

After iterating: six. (pause) Not ten. It didn't walk the whole vector. But not four either. Here's why. To get the first result, filter tests 1, 2: two calls, yields 4. To get the second, it tests 3, 4: four calls, yields 16. Now `take` has its two. But the loop's increment runs first: `++` advances the filter to the *next* match, so it tests 5 and 6, before `take` checks its counter and says "stop". Six.

Lazy means "no more than needed", plus a small lookahead. It doesn't mean "exactly once per element you see." That matters the moment your predicate has a side effect, or is expensive, or reads hardware. So rule of thumb: keep predicates pure and cheap. If it logs, counts, or touches a register, a view will surprise you.

IF AHEAD: The same lookahead happens with `take_while`, and it's worse with `istream` views: one extra element gets consumed from the stream. If you're reading

a framed protocol and stop with `take(n)` on an `istream_view`, the next frame's first token may already be gone. That's a real bug shape, and it's why "consumed once" is part of the guidance on slide 46.

19. The pipe syntax • C++20 • target 0:38, ~2 min

This is `top_n_by_value` from the exercise, task 2, before and after.

Left: declare `matching`, `copy_if` with a `back_inserter` and a lambda, `stable_sort` with a comparator lambda, `resize`. Ten lines, and you have to read all of them to see that it's three steps.

Right: `records | filter(same sensor) | to<std::vector>()`. Then `stable_sort(matching, greater{}, &Record::value)`. Then the same `resize`. **The pipeline reads top to bottom as the steps.**

The mechanics are simple: `range | adaptor` is just `adaptor(range)`. The pipe is function application written left to right.

Notice the filter lambda is still there. That's not a member comparison; it captures `sensor`. Projections removed the comparator lambdas, not the predicates. (beat)

And notice we didn't sort the view. We materialized into a vector first. Why?

ASK: "Why `to<vector>` before the sort instead of sorting the filtered view directly?"
Expected: filter is only bidirectional, sort needs random access; and sorting through a view would reorder the original records. Say: "Both. The concepts would reject it anyway."

IF BEHIND: "`copy_if` plus `back_inserter` becomes `filter | to<vector>`.
Pipe is just function application, left to right."

20. The core adaptors • C++20 • target 0:40, ~2 min

The tour. Twelve adaptors on screen plus a chain at the bottom. Two it doesn't show: `elements`, which is keys and values generalized to any tuple index, and `common`, which we'll meet on slide 27.

DO: Run `./build/demos/s04/demo_s04_adaptors_tour`. Each line prints its label and result. Point at three: `drop_while 4 5 6`, `join a b c d` (a vector of strings joined into one range of characters), and `chain 6 5`. Recovery: the expected output is in the comments on each line of the file; scroll and read them.

filter and transform are eighty percent of real use. take, drop, and their _while forms are most of the rest. (beat)

Look at the last line, `chain`. `Reverse`, then filter greater than two, then take two: `6 5`. Read it like a sentence. That's the payoff.

And look at `split` on line 11. Split the string on commas, then transform each piece into a `string_view`. That's exactly the shape of the exercise's `split` function, which is the next slide.

IF AHEAD: `views::keys` and `views::values` work on anything whose elements are pair-like, not only maps. A vector of `std::pair` or `std::tuple` works too, and `views::elements<2>` pulls the third field. For a vector of plain structs, though, you want `transform(&Record::sensor)`: structs aren't tuple-like unless you specialize `tuple_size`.

IF BEHIND: "Filter and transform are most of it. Take, drop, and the `_while` forms are the rest."

21. `views::split` and the edge case the tests caught • C++20 • target 0:42, ~2 min

This is the exercise's `split`, task 3, as a pipeline. `text | split(delimiter) | transform(to_string_view) | to<std::vector>()`.

What does `split` yield? Subranges of the original characters. The `transform` turns each one into a `string_view` over the same buffer. **No copies. Every field is a window into the caller's string.** Remember that sentence in two slides.

Now the first line of the function. `if (text.empty()) return {std::string_view{}};` Why is that there?

Because when the exercise's loop became this pipeline, one test failed. `split("", ',').size() == 1`. The old loop returned one empty field for an empty string.

`views::split` of an empty string yields zero pieces. (pause)

Which is right? Honestly, either. One empty field is how CSV readers usually behave. Zero fields is how `views::split` behaves. Both are defensible.

What isn't defensible is changing it silently. **The tests define the contract, not the pipeline.** The solution kept the old contract with that early return.

You'll hit this in class, task 3. Let it fail. That's the point of the task.

IF AHEAD: Trailing delimiters are another edge: `"a,b,"` gives three pieces with `views::split` in C++23 and in the C++20 DR (P2210), the last one empty. But the pre-DR implementations in GCC 10 and 11 differed in details. If you have a test suite from an older toolchain and it suddenly diverges on an upgrade, check whether your `split` behavior was what you thought. Also, `views::lazy_split` exists; it's the original C++20 design that works on input ranges but yields awkward inner ranges you can't turn into a `string_view`. For strings, use `split`.

IF BEHIND: "`views::split("")` gives zero pieces; the loop gave one. The test caught it. Tests define the contract."

22. View semantics • C++20 • target 0:44, ~2 min

A view is a `string_view` over a computation. Non-owning, O(1) to copy, holds a reference to its source plus the adaptor's state.

First bullet: `v | filter(p)` holds a `ref_view` of the vector, which is a pointer, plus `p`. Copying the view copies a pointer and a lambda.

Second bullet, the exception: `std::move(v) | filter(p)`. Now the source is an rvalue, so the vector is moved into an `owning_view`, inside the view. That was a C++20 defect report, P2415, and all our compilers have it. It's what made line 2 on slide 12 safe: an rvalue container piped into a view gets owned by the view.

Third bullet. **A view must not be stored past the range it refers to.** Same rule as `string_view` and `span`. If you wouldn't store a `string_view` to it, don't store a view to it.

Fourth bullet: the formal view concept. Movable, O(1) destruction, and the "cheap to copy if copyable" rule. It used to require default construction too; that was dropped in another DR, which is what the slide's awkward wording means. `views::all` is what the pipe applies to your container to turn it into a view: `ref_view` for lvalues, `owning_view` for rvalues.

ASK: "Can you put a filter view in a class member?" Expected: technically yes, but its type includes a lambda type, and it holds a pointer to the source, so the class must not outlive or move away from the source. Say: "Right. And if the class is copied, the copy's view still points at the original's vector. That's the bug. Store the container, or store a `span`, and build the view in the function that uses it."

IF BEHIND: "A view is a `string_view` over a computation: cheap, non-owning, don't store it past its source. Rvalue containers get moved into the view."

23. The `const` view trap • C++20 • target 0:46, ~3 min

Slow down. This one goes against twenty years of training.

A `const filter_view` is not a range. You can't iterate it. (pause)

Here's the reason. The range concept requires `begin()` to be amortized O(1). But `filter_view::begin()` has to find the first element that matches, which might be a long scan. So it does the scan once and caches the result inside the view. Caching means writing to a member. Writing means `begin()` can't be `const`. So `const filter_view` has no `begin()`.

Look at the two function templates. `count_const` takes `const R&`, the way we've all been taught to take "a thing I only read". `count` takes `R&&`, a forwarding reference. Same body.

DO: In the Compiler Explorer tab with `const_view_trap.cpp`, make sure - `DSHOW_ERRORS` is in the flags (or locally: `g++ -std=c++23 -DSHOW_ERRORS -c demos/s04/const_view_trap.cpp`). Expected GCC error on line 14: passing 'const `std::ranges::filter_view<std::ranges::ref_view<std::vector<int>>, ...>`' as 'this' argument discards qualifiers. Point at `const` and `filter_view` in that message. Then remove the flag: it builds, and `./build/demos/s04/demo_s04_const_view_trap` prints 2 then 4. Recovery: if Compiler Explorer is slow, read the error text from here.

So the exercise hit this in `serialize`. A `serialize(const R&)` overload, called with `v | filter(...)`, didn't compile. Changing it to `serialize(R&&)` fixed it. That's at-home task 4.

Take ranges by `R&&` in generic code, and iterate with `auto&&`. That's exactly how every `std::ranges` algorithm takes its range.

Which views have the problem? Bottom of the slide. Const-iterable: `transform`, `take`, `drop` over random access, `iota`, `all`. Not const-iterable: `filter`, `drop_while`, `split`, `chunk_by`, and `join` in some cases. The pattern: anything whose `begin()` has to search.

ASK: "Does R&R mean the function can modify the caller's container?" Expected: it can, if the caller passes a non-const lvalue. Say: "Yes. R&R gives up the `const` promise. That's the trade. If you need the promise for a container, overload, or document it. For views, the promise was never available."

IF AHEAD: Two follow-ons. First, the cache is also a thread-safety issue: calling `begin()` on the same non-const `filter_view` from two threads is a data race, because both may write the cache. Share the source, not the view; give each thread its own view. Second, C++23 has `views::as_const`, which makes the *elements* `const`, not the view. It doesn't fix this. It's for "I want to iterate without being able to modify the source", the `std::as_const` idea for ranges.

24. Dangling with views • C++20 • target 0:49, ~2 min

Slow down. This one ships.

Views didn't change the lifetime rules from Session 2. They made them easier to break. (pause)

Line 1 in `dangling.cpp`: `auto fields = split(std::string("a,b"), ',');` The `split` function takes a `string_view`. A temporary `std::string` converts to a `string_view` happily. `Split` returns a vector of `string_views`, each pointing into that temporary string's buffer. At the semicolon, the temporary dies.

It compiles. No warning. And `fields[0]` on the next line is undefined behavior. (pause)

It compiles, it runs, and in a small test it might even print the right answer, because the freed memory hasn't been reused yet. That's what makes it dangerous. It passes your unit test and fails in the field.

This is not the dangling type from slide 14. That protection only works when the algorithm can see the temporary. Here the temporary was consumed inside `split`; what comes out is a vector, which is a perfectly normal owned value. It just contains pointers to nowhere. Clang's `-Wdangling` doesn't catch this one either.

Line 3, `split(line, ',')`: `line` is a named string that outlives the `fields`. Fine.

Line 4, the fix when the result must outlive the source: `materialize`. `transform` each `string_view` into a `std::string`, then `to<std::vector>()`. Notice this is safe even though it starts from a temporary, because the whole chain runs before the semicolon. The temporary string is still alive while the copies are made.

ASK: "How would you catch line 1 in CI?" Expected: AddressSanitizer. Say: "Yes. ASan with a test that actually reads the field reports heap-use-after-free. That's the strongest argument for running your unit tests under ASan on every build. Static warnings won't find this."

At-home task 8 makes you write this bug on purpose.

IF AHEAD: An API design heuristic for review: a function that takes `std::string_view` and returns something containing views into it (a vector of `string_view`, a span, a view) should either be documented "result borrows from the argument" or take `const std::string&` and delete the rvalue overload: `split(std::string&&, char) = delete;`. That turns line 1 into a compile error. It's the same trick the standard uses for `std::ref` on temporaries.

25. The cost model • C++20 • target 0:51, ~3 min

The performance question. Is this slower than a loop?

Most pipelines inline to the loop you would have written. Some don't. (beat)

First function, `sum_even_squares`: filter evens, transform to squares, sum in a range-for. At -O2, on both compilers, that's one loop with a branch. Same as the hand-written loop.

DO: Switch to the Compiler Explorer tab with `cost_model.cpp`, GCC 14 -O2 -std=c++23. Point at the `sum_even_squares` body: one loop, a test of the low bit, a multiply, an add. No calls. If the Clang pane is open, point at it: same shape. Recovery: if the tab won't load, say "one loop, no calls, I'll leave the link in the chat" and move on.

Second function, `reverse_of_filter`. Filter, then reverse. Here's the cost: the filter predicate runs about twice per element. Put a counter in the predicate, like the lazy demo: six elements, a forward filter runs the predicate six times, reverse of filter runs it twelve.

Why? The slide says it's because reverse has to find the end. For a vector, that's not quite it: a filter over a vector already knows its end. The real reason is how `reverse` iterates. It uses `std::reverse_iterator`, and every time you dereference a reverse iterator, it copies the underlying iterator and steps it back one. Stepping a filter iterator back means scanning backward, running the predicate, to the previous match. Then ++ on the reverse iterator does that same scan again. **So every element pays for the backward search twice.** (pause) The slide's explanation is right for a different case: a filter over a range whose end is a sentinel, where reverse first has to walk the whole thing to find the end.

Third: `join` and `split` are forward-only. No `size()`, no random access, a branch per element. Fine for parsing a config line. Measure before you put them in a hot loop.

DO: Optionally run `./build/demos/s04/demo_s04_cost_model`. Expected: 56 12. Same answers; the cost difference is invisible at this size, which is the point: you only see it with a profiler.

The rule: views in code you own, and measure the hot paths. A plain loop is still the right answer in an inner loop where the view would do extra work.

IF AHEAD: Debug builds are the other cost nobody mentions. At -O0, every adaptor layer is a real function call, and a four-stage pipeline can be several times slower than the loop. If your team ships or tests debug builds with timing requirements

(hardware-in-the-loop rigs often run unoptimized for debuggability), that matters. -
Og recovers most of it. And stepping through a pipeline in GDB is painful; skip -rfu
^std:: in your .gdbinit helps.

26. Composable pieces • C++20, C++23 • target 0:54, ~2 min

An adaptor closure is a value. Name it, store it, compose it.

Top snippet, adaptor_closure.cpp.inline constexpr auto evens =
views::filter(...). No range attached. Same for squared. Then top2_even_squares
= evens | squared | take(2). You just composed three adaptors into one, still without
any data. That's a reusable, testable pipeline fragment. You can unit-test it against a fixed
vector and then use it everywhere.

Bottom snippet, C++23. Want your own function to work after a pipe? Derive from
std::ranges::range_adaptor_closure<sum_fn>. That's CRTP: the base gets the
derived type as a template argument. Now v | evens | sum works, and sum is your function
object.

DO: Run ./build/demos/s04/demo_s04_adaptor_closure. Expected: 4 16 |
12. The first two are the stored pipeline; 12 is 2 plus 4 plus 6 through your own sum.
On libc++ 18 the second half prints (range_adaptor_closure not
available); it arrives in libc++ 19.

Notice that storing a closure is safe, unlike storing a view, because a closure doesn't refer to any
data. (beat) That's the line to remember: closures are fine to store, views aren't.

IF BEHIND: "Adaptor closures are values you can name and compose. C++23 lets you
make your own pipeable."

27. std::ranges::to • C++23 • target 0:56, ~2 min

**The single most-missed piece of C++20 ranges: there was no clean way to turn a view
back into a container. (pause)**

Left: C++20. std::vector<int> a(evens.begin(), evens.end()). That works only if
the view is "common", meaning begin and end have the same type. A filter over a vector
happens to be common, so line 2 compiles. Plenty of views aren't, iota without a bound or
anything ending in take_while, and then you have to adapt with views::common first. Two
extra lines, and a constructor that can't reserve.

Right: C++23. | std::ranges::to<std::vector>(). You don't even name the element
type; it's deduced. Line 2: to<std::set> dedupes and sorts in one step. Line 3: a range of
pairs goes straight into to<std::map>. Line 4: a vector of vectors of char becomes a vector of
strings: to converts element-wise, recursively.

Don't forget the parentheses: to<std::vector>(). Without them it's a template, not an
adaptor, and the error is not friendly.

This is the tool for the rule from slide 24: **when the result must outlive the source,
materialize with to.**

DO: `./build/demos/s04/demo_s04_ranges_to`. Expected: 1 1 3 4 16 ab.
One even, one even, three greater than one, four distinct values in the set, `m[4]` is 16,
and the first word is ab. Recovery: read it.

IF AHEAD: `to` reserves when it can. If the source is a `sized_range` and the container has `reserve`, it calls it. A filter isn't sized, so it can't; a `transform` over a vector is, so it does. And containers gained `from_range_t` constructors and `insert_range/append_range` members in C++23 (P1206), which `to` uses when present. Availability: `to` is GCC 14 and libc++ 17.

28. `zip`, `enumerate`, and indexing · C++23 · target 0:58, ~1 min

The report needs a rank column: one, two, three. Normally that's a counter variable next to the loop.

Line 2: `views::zip(names, values)` walks two ranges in lockstep and gives you tuples you can unpack with a structured binding. Line 6: `zip(iota(1uz))` with `names`. `iota` with no end is infinite, but `zip` stops at the shortest range, so you get exactly as many indices as `names`, starting at 1. **That's the portable way to number things, and it's what the exercise uses.**

`views::enumerate` is the intended spelling. It starts at zero. libc++ 18 doesn't have it, hence the `#ifdef`.

DO: `./build/demos/s04/demo_s04_zip_enumerate`. On GCC 14: four lines, ending with `rpm:4811 temp:42 pressure:101` from `zip_transform`. On libc++ 18 you'll see only the first two lines. Recovery: read the first two lines from the comments.

IF BEHIND: "`zip(iota(1uz), xs)` numbers anything, portably. `enumerate` is the C++23 spelling, missing on libc++ 18."

29. Views takeaway · target 0:59, ~1 min

Views for transformations that are consumed once, lazily. to when the result must outlive the source. Take ranges by R&&. Never store a view past its source.

And the two traps the exercise's tests caught: `views::split("")` and the `const filter_view`. Views have their own semantics. The tests define the contract. (pause)

Monday morning: turn one `copy_if` plus `back_inserter` into `filter | to<vector>`.

It's 1:00. Let's see what C++23 added.

30. What C++20 was missing · target 1:00, ~2 min

Here's the honest history. C++20 ranges shipped incomplete. The committee ran out of time. Look at the left column: no way to materialize a view, no `zip`, no `enumerate`, no group-by, no windows, no fold, no `contains`, no join with a separator.

The plan for C++23 was written before C++20 was even published. P2214, "A Plan for C++23 Ranges", listed every one of these gaps, prioritized them, and C++23 delivered them.

So if you looked at ranges in 2020 and decided they were a toy, you weren't wrong for 2020. You'd be wrong now. **C++23 is the first standard where views are practical for daily code.** (beat)

ASK: "Which of these rows would you have used last week?" Expected: `to`, `contains`, and `zip` usually come up; telemetry people say `slide` or `adjacent`. Say: "Good, the next three slides are those."

IF AHEAD: P2214 tiered the work: tier 1 for C++23, tier 2 and 3 deferred. Tier 2 items like `views::concat` and `views::cache_latest` landed in C++26, which is slide 44's preview. The paper is very readable and is the best single source for "why does ranges look like this".

IF BEHIND: "C++20 ranges shipped without `to`, `zip`, `enumerate`, `chunk_by`, or `fold`. C++23 filled them in."

31. `views::chunk_by`: the group-by • C++23 • target 1:02, ~4 min

This is the report's group-by-sensor, exercise task 5.

Left, the C++11 way (strictly it's C++17, with that structured binding, but it's the shape you know). Build a `std::map` from sensor name to a vector of records. Every record gets copied into a map node's vector. Then iterate the map, sort each vector by value descending with a comparator lambda, print the front.

That's one heap allocation for each map node, more for each vector's growth, and a copy of every record.

Right, C++23. Two steps. Sort the records by sensor, with a projection. Then `views::chunk_by` with a predicate "same sensor as the previous one". **`chunk_by` splits a range into a subrange for every run of consecutive elements where the predicate holds.** (pause) Because we sorted first, every run is one sensor.

Each group is itself a range, a subrange of the original vector. No copies, no allocation per group. So inside the loop, the per-group code is just the algorithms you already know: `ranges::max(group, {}, &Record::value)` gets the best reading.

The important word is *consecutive*. **`chunk_by` only groups adjacent elements.** If you forget the sort, you get one group per run, so `rpm`, `temp`, `rpm`, `temp` would print four groups, not two.

DO: `./build/demos/s04/demo_s04_chunk_by`. Expected four lines: `rpm: 4830`, `temp: 42.5`, then the same two again. The first pair is the map version, the second is `chunk_by`. Same answer. Then the experiment: comment out the `ranges::sort` line in `top_per_sensor`, rebuild (`cmake --build build --target demo_s04_chunk_by`), and run: the `chunk_by` half now prints four groups, `rpm: 4800`, `temp: 41`, `rpm: 4830`, `temp: 42.5`. Put the sort back. Recovery: if the rebuild drags, just say what it prints.

ASK: "The map version's output is sorted by sensor name. Is the `chunk_by` version guaranteed to match?" Expected: yes, because we sorted by sensor; `std::map` orders by `std::string's <` and so does the projection. Say: "Exactly the kind of thing the

report-diff test checks. If the map had a custom comparator, you'd need the same one in the sort."

IF AHEAD: The predicate gets two adjacent elements, so `chunk_by` can do more than equal keys. `chunk_by([])(auto a, auto b) { return b.ts - a.ts < gap; }` splits a telemetry stream into bursts wherever the timestamp gap exceeds a threshold. That's a session detector in one line. Also, `chunk_by` caches its first chunk like `filter` does, so it's on the not-const-iterable list from slide 23.

32. Windows: `chunk`, `slide`, `stride`, `adjacent` • C++23 • target 1:06, ~3 min

Four views for signals. This is the telemetry slide.

`signal` is 1, 2, 4, 8, 16, 32, 64: powers of two, so the outputs are easy to check by eye.

`chunk(3)`: non-overlapping blocks of three. The last block gets what's left: `[64 .. 64]`.

`slide(3)`: overlapping windows of three. One, two, four; then two, four, eight; and so on.

`slide(3)` plus an average is a moving average in three lines. No index arithmetic, no off-by-one at the end.

`stride(2)`: every second element. That's decimation.

`adjacent<2>`: pairs of neighbors, as tuples you can unpack. `b - a` gives deltas. `pairwise` is just the name for `adjacent<2>`; the last line uses it to print ratios.

```
DO: ./build/demos/s04/demo_s04_windows. Expected on GCC 14: [1 .. 4]
[8 .. 32] [64 .. 64] 2.3 4.7 9.3 18.7 37.3 1 4 16 64 1 2 4 8 16
32 2 2 2 2 2 2 Point at the moving average line, then the deltas. On libc++ 18 it
prints window views not available in this standard library; none of
these are in libc++ until 20. Recovery: read the five lines from here.
```

ASK: "Seven samples, window of three. How many moving-average outputs?"

Expected: five. Say: "N minus W plus one. And `slide` gets that right for you; the hand-written version is where the off-by-one lives."

These aren't exotic. Deltas, decimation, and moving averages are what every telemetry pipeline does first. (beat)

IF AHEAD: `chunk` and `slide` yield subranges, so the window is a view into the original data, no copy. `adjacent<N>` yields tuples of references, which is why it takes N as a compile-time template argument and `slide` takes a run-time size. Pick `adjacent` when N is small and fixed, because structured bindings work; pick `slide` when N comes from configuration.

IF BEHIND: "Chunk is blocks, slide is overlapping windows, stride decimates, adjacent gives neighbors. A moving average is `slide` plus a mean."

33. `cartesian_product`, `repeat`, `join_with`, `as_rvalue` • C++23 • target 1:09, ~3 min

The rest of the C++23 view family. Four lines in `zip_family.cpp`.

`cartesian_product(sensors, channels)`: every combination. For a test matrix, sensor by channel by mode, that's nested loops collapsed into one. Output: rpm/1 rpm/2 temp/1 temp/2.

`repeat(7, 3)`: seven, three times. With no count, it's infinite, which pairs nicely with `zip` for "a constant alongside every element".

`join_with`: the one people will actually use. **String joining with a separator, finally standard.** `sensors | join_with(std::string_view{"", "}"` gives rpm, temp. Watch the separator: pass a `string_view`, not a bare string literal. A literal decays to `const char*`, which isn't a range, and the error is long. A single char also works: `join_with(' ', ')`.

`as_rvalue`: turns each element into an rvalue so the consumer moves instead of copying. Here `to<vector>` moves the strings out of `src`.

DO: `./build/demos/s04/demo_s04_zip_family`. Expected on GCC 14: rpm/1 rpm/2 temp/1 temp/2, then 7 7 7, then rpm, temp, then 2 true. On libc++ 18 it prints C++23 view family not available in this standard library. Recovery: read them.

About that `true` at the end: it says the source string is empty after the move. On our libraries it is, for short strings. **The standard only says moved-from is valid but unspecified.** So the demo is showing you what happens, not a guarantee. Don't write code that depends on it.

IF AHEAD: `as_const` is the sibling the outline mentions: `src | views::as_const` makes the elements `const` references so a consumer can't modify the source. Remember from slide 23 it does not make the view `const-iterable`. And `cartesian_product`'s size is the product of the sizes, which overflows faster than you'd think for test matrices: four ranges of a thousand is 10 to the 12th.

IF BEHIND: "Cartesian product for every combination, `join_with` for joining strings with a separator, `as_rvalue` to move out."

34. `ranges::fold_left` and friends • C++23 • target 1:12, ~3 min

C++20 ranges had no `accumulate`. C++23 has `fold`s. (beat)

Line 2 of `fold.cpp`: `fold_left(v | transform(&Record::value), 0.0, std::plus{ })`. A range, an initial value, a binary operation. Notice the `transform` with a member pointer: that's how you get a projection into a fold, because the folds don't take a projection argument.

Line 5: `fold_left_first`. No initial value; it uses the first element. So what does it return on an empty range? (pause) There is no first element. So it returns `std::optional`. Empty range, `nullopt`. The type system makes you handle it.

Line 9: `fold_right`. It associates from the right, which matters when the operation isn't commutative. Look at what it does to string concatenation: the records are rpm, rpm, temp, and the output is temp; rpm; rpm;. Reversed. Fold right processes the last element first.

DO: ./build/demos/s04/demo_s04_fold. Expected on GCC 14: 8 4
temp; rpm; rpm;. Total 8, max 4, and the reversed names. On libc++ 18 the last field
prints (fold_right not available). Recovery: read it.

Now the bug at the bottom. `std::accumulate(v.begin(), v.end(), 0)` on doubles. The type of the init decides the accumulator type, so 0, an int, truncates at every step. 1.5 plus 2.5 becomes 1 plus 2. **That's a classic bug and it compiles silently, maybe with a conversion warning if you have -Wconversion on.**

Here's the part the slide doesn't say. `fold_left` fixes it even if you pass 0. Its accumulator type is whatever the operation returns, and `std::plus{}` with an int and a double returns a double. So `fold_left` of 1.5 and 2.5 with init 0 returns 4.0, a double. `fold_left_first` sidesteps it entirely because there's no init.

ASK: "Can you pass a `std::execution::par` to `fold_left`?" Expected: no. Say: "Right. Folds are sequential by definition: left fold has an order. For parallel, it's `std::reduce` with a policy, which is slide 41."

IF AHEAD: The other two folds: `fold_left_with_iter` and `fold_left_first_with_iter` return both the result and the end iterator, for when the range is input-only and you need to know where you stopped. And note `std::plus{}` with empty angle brackets: that's the transparent version from C++14. `std::plus<int>{}` would bring the truncation right back.

35. contains, starts_with, find_last, iota • C++23 • target 1:15, ~3 min

Small algorithms that each delete an idiom.

Line 1: `ranges::contains(v, 3)`. **No more `find(...)` != `end()`.** It reads like what it means.

Line 2: `contains` with a projection: does the sensor table contain a config named "temp"? That's the exercise's `validate()` duplicate check, task 6. And `contains` is `constexpr`, so the `static_assert` that validates the table at compile time still works. That connects straight back to Session 3.

Lines 4 and 5: `starts_with` and `ends_with` on any range, not just strings. Line 9: `find_last` returns a subrange from the last match to the end. Line 10: `ranges::iota` fills a range with 10, 11, 12, and so on.

DO: ./build/demos/s04/demo_s04_cpp23_algorithms. Expected on GCC 14:
true true false false 3 14. Before anyone asks: the two false's are not the answers. `starts_with` and `ends_with` aren't in `libstdc++` until GCC 15, so that block is compiled out and the fallback sets them to false. On libc++ 18 those two print true. The 3 is the index of the last 2, and 14 is the last value `iota` wrote.
Recovery: say exactly that.

This is what the support matrix looks like in real life: the same file, two compilers, different holes. (pause) Feature-test macros like `__cpp_lib_ranges_starts_ends_with` are how you write code that builds on both.

One correction on availability: the slide notes say `find_last` and `ranges::iota` arrived in GCC 14. They're actually in `libstdc++ 13`; the feature-test macros are defined there. On our GCC 14 baseline it makes no difference.

`ranges::shift_left` and `shift_right` are the last ones; they're only in the newest libraries. The C++20 `std::shift_left` with iterators is everywhere.

IF AHEAD: Prefer feature-test macros over version checks when you can. This file uses both: `__cpp_lib_ranges_starts_ends_with` for one block, and a `_LIBCPP_VERSION >= 190000` check for `find_last`, because the version check is clearer when the gap is a whole library release. Under `-Werror`, an unused variable in the fallback branch will break the build; that's why the demo sets `pre` and `suf` in both branches.

IF BEHIND: "contains replaces `find != end`, and works with projections and in `constexpr`. The rest are small; check the matrix."

36. Formatting and generating ranges • C++23 • target 1:18, ~2 min

Two things to close the C++23 block.

Line 1: range formatting from Session 2 applies to views. `println("{} ", v | take(3))` prints `[1, 2, 3]`, brackets and commas included. Line 2: `{: :.1f}` applies a format spec to each element: the first colon starts the range spec, the second the element spec. Range formatting is `libc++ 17` and `GCC 15`, so on `GCC 14` today, it's a loop.

Then the preview. `std::generator<Record>`. A coroutine: it reads lines, parses each, and `co_yields` the good ones. **A generator is an input range, so it plugs straight into a pipeline.** Line 7 filters it like any other range.

That means the exercise's `load_stream`, which reads the whole file into a vector, could become a generator that feeds the pipeline one record at a time, without holding the file in memory. That's Session 5. `GCC 14` has `std::generator`; `libc++` doesn't yet.

IF BEHIND: "Views format with `println`. `std::generator` is a coroutine that's also a range; that's next session."

That's the C++23 block. 1:20. Now let's make things go faster with one extra argument.

37. Execution policies • C++17 • target 1:20, ~4 min

C++17 added a first argument to most of the algorithms: an execution policy. **Same algorithm, one extra argument, and the library is permitted to run it in parallel.** (beat) "Permitted", not "required".

Four policies. `seq`: as if sequential. `par`: may split the work across threads. `par_unseq`: may also interleave iterations within one thread, which means vectorize. And C++20 added `unseq`: vectorize only, one thread.

Look at `parallel.cpp`. Five million doubles, shuffled. Sort with `seq`, reshuffle, sort with `par`, then reduce with `par_unseq`.

DO: `./build/demos/s04/demo_s04_parallel`. Expected shape: seq NNNms
par NNNms par_unseq reduce NNms sum 1.250e+13. Before you read the
numbers aloud, look at lines 27 to 29 of the file: `t1` is captured after the seq sort, then
the vector is reshuffled, then `t1` is captured *again*. So the printed seq time includes
the second shuffle, roughly 70 ms for five million doubles. Say so: "The seq number is
inflated by about the cost of a shuffle; that's a bug in the demo, and it makes par look
better than it is." If `ldd` showed no TBB, par will be about seq minus that shuffle.
Recovery: if the binary prints `parallel algorithms not available in this
standard library build`, you're on `libc++ without -fexperimental-
library`; talk through the code.

And there's the honest lesson in that demo bug: **benchmark code gets less review than
production code, and that's where wrong conclusions come from.** (pause)

The last comment on the slide: none of this works with `std::ranges::algorithms`. Execution
policies are iterator-pair only until C++26.

ASK: "If you parallelize a `std::reduce` over doubles, will you get the same sum
every run?" Expected: not necessarily. Say: "Right. Floating-point addition isn't
associative, and reduce may add in any order. That's slide 39."

IF AHEAD: `std::reduce` with `par_unseq` over a sorted sequence of doubles is the
fastest line in the demo because summation is trivially vectorizable. The interesting
comparison would be `std::accumulate` over the same data with `-O2`: GCC won't
vectorize the accumulate loop by default, because reordering floating-point adds
changes results. `-ffast-math` would let it, which is exactly the permission reduce
gives you explicitly, for one call, without a global flag. That's a much better thing to
approve in review than `-ffast-math`.

38. What actually runs in parallel • target 1:24, ~2 min

Here's the table that saves you a week.

`libstdc++`, GCC: the parallel backend is Intel TBB. You need the TBB headers when you compile
and `-ltbb` when you link. **Without TBB, par compiles, runs, and is sequential.** No
warning.

`libc++`, Clang: its own backend, experimental, behind `-fexperimental-library` on 17 and
18. Without that flag, the feature-test macro `__cpp_lib_parallel_algorithm` isn't defined
at all, which is why the demo checks it. And depending on how your `libc++` was built, the
default backend may itself be serial; check before you trust a number.

MSVC: Windows thread pool, works out of the box.

The repo's Dockerfile installs TBB, and CMake links it when `find_package(TBB)` succeeds.

**The trap: a team turns on par everywhere, sees no speedup, and concludes parallel
algorithms are useless, when the binary was never linked against TBB.** (pause)

IF AHEAD: For a certified or air-gapped toolchain, TBB is a third-party dependency
with its own version, license (Apache 2.0), and supply-chain review. That's often the

real blocker, not the C++ standard. If TBB isn't approvable, `par` on GCC is a no-op, and you should say so in the design rather than leave `par` in the code implying parallelism that never happens.

IF BEHIND: "On GCC, no TBB means no parallelism, silently. Check `ldd`."

39. Your responsibilities with `par` • target 1:26, ~3 min

The library gives you permission to run in parallel. You give it a promise in return.

No data races between elements. The body may not touch shared state without synchronization. With `par_unseq` it's stronger: not even with a mutex. Iterations can interleave on one thread, so if one iteration takes a lock and another on the same thread tries to take it, you deadlock. No locks, no allocation (the allocator may lock), no I/O.

Exceptions terminate. An exception escaping the body under any parallel policy calls `std::terminate`. No error propagates. In a codebase that uses exceptions for error handling, that alone can rule `par` out of a function.

Order is unspecified. `for_each(par)` visits in any order. `reduce` adds in any order, so a floating-point sum can differ in the last bits from run to run. If you have regression tests that compare floating-point output exactly, a `par` reduce will make them flaky.

When is it worth it? Large N, tens of thousands or more, or expensive per-element work. Starting work on a thread pool costs microseconds. **A `par` sort of a thousand ints is slower than `seq`.**

And measure. On a two-core box, `par` sort is barely faster than `seq`.

ASK: "You have `for_each(par, ...)` and the body does `results.push_back(x)` on a shared vector. What's wrong?" Expected: data race on the vector. Say: "Right. Fix it by sizing the output first and writing by index with `transform(par, ...)`, so each element owns its own slot."

IF BEHIND: "No shared state, exceptions terminate, order is unspecified, and only worth it for big N. Measure."

40. Parallel takeaway • target 1:29, ~1 min

Execution policies are the cheapest parallelism in C++: one argument, if the body is independent per element.

They're also the easiest to misuse. No TBB means no parallelism on GCC. `par_unseq` forbids locks. (beat)

Monday morning: check `__cpp_lib_parallel_algorithm` and `ldd` before you trust a `par` benchmark. And read the timing code as carefully as the code being timed.

Anything more complicated than "do this to every element independently" is Session 5's concurrency material. Now, ten minutes of algorithm additions you may have missed.

41. C++17 numerics • C++17 • target 1:30, ~3 min

The numeric algorithms that came with the execution policies.

Line 1: `std::reduce`. Like `accumulate`, with two differences. It may reorder the additions, which is what lets it parallelize. And its init defaults to the value type, `double{} here`, so it can't truncate the way `accumulate(..., 0)` does. **For a plain sum of doubles in new code, `reduce` is the better default, unless you need a specific order.**

Line 3: `transform_reduce` with two ranges: the sum of products. Dot product, weighted sum, one pass, parallelizable.

Lines 5 and 6: scans, prefix sums. `inclusive_scan` gives 1.5, 4.0, 7.0: each element includes itself. `exclusive_scan` starts from the init and excludes the current element: 0, 1.5, 4.0. Cumulative distance from speed samples, cumulative counts for a histogram: that's a scan.

Then `std::sample`: five elements without replacement, with an explicit engine so it's reproducible. And `std::clamp`.

DO: `./build/demos/s04/demo_s04_numeric17`. Expected: 7 17.5 4 5 100.
Sum 7, dot product 17.5, then 4, which is running [2] after the exclusive scan overwrote the inclusive one, then five samples, and the clamp. Recovery: read it.

IF AHEAD: `std::clamp` returns a reference to one of its arguments. `const int& c = std::clamp(x, 0, 100);` with literals binds to a dead temporary when the result is a bound. It's in the same family as the dangling bugs today. Take the result by value. Also, `clamp` requires `lo <= hi`; violating it is undefined, not "swapped".

IF BEHIND: "reduce is the reorderable `accumulate` with a safe init;
`transform_reduce` is a dot product; scans are prefix sums."

42. C++17 searchers and `for_each_n` • C++17 • target 1:33, ~1 min

Thirty seconds. `std::search` with a searcher object. **Build a `boyer_moore_searcher` once for a pattern, and search many buffers with it.** That's log scanning, or finding a sync word in a protocol stream. The preprocessing cost is paid once.

Three flavors: `default_searcher`, `boyer_moore_searcher`, and `boyer_moore_horspool_searcher`, which uses less memory.

And `for_each_n`: the first `n` elements, without computing an end iterator. (beat)

IF BEHIND: Fold into slide 41 (cut list). Say: "C++17 also has Boyer-Moore searchers for repeated pattern search, and `for_each_n`."

43. C++20 algorithm additions • C++20 • target 1:34, ~2 min

Fast list.

`shift_left` and `shift_right`: move elements by `n` within a range. **`shift_left` is the one people didn't know they needed: "drop the oldest `n`" on a fixed buffer, without `rotate`'s extra work.**

`lexicographical_compare_three_way`: the spaceship for sequences.

`std::ranges::` versions of the `<algorithm>` functions. Not quite every one: `shift_left`, `shift_right`, and `iota` only got `ranges::` versions in C++23, as the next slide shows, and `lexicographical_compare_three_way` has none. And nothing from `<numeric>`.

`midpoint` and `lerp` from Session 2. `midpoint` is the overflow-safe average; it's the fix for the binary-search bug where $(lo + hi) / 2$ overflows. `ssize` for a signed size. `std::erase` and `erase_if` for every container: the end of erase-remove. And the `unseq` policy.

IF AHEAD: `std::midpoint` on integers rounds toward the first argument, not toward zero. `midpoint(0, 3)` is 1, `midpoint(3, 0)` is 2. That asymmetry is deliberate and documented, and it surprises people in review.

IF BEHIND: "Shift, three-way lexicographic compare, `midpoint`, `ssize`, and `erase_if`."

44. C++23 algorithms and what comes next • target 1:36, ~2 min

The availability table for the C++23 algorithms. I won't read it. Two corrections and one preview.

The corrections: the `find_last` and `ranges::iota` rows say GCC 14. They're in libstdc++ 13. Not that it matters on our GCC 14 baseline, but if you're on 13, you have them.

The preview is the last row. **C++26 is where ranges and parallelism meet:**

`std::ranges::sort(par, v)`, a policy as the first argument of the range algorithms. That answers the question someone always asks: "why can't I use `par` with ranges?" Answer: you will, in C++26.

Also in C++26: `views::concat`, to chain ranges end to end; `views::cache_latest`, which fixes a cousin of the reverse-of-filter cost (a `transform` followed by a `filter` calls the transform twice per passing element); and `ranges::generate_random`.

ASK: "Given your toolchain upgrade cadence, when will you see C++26 library features?" Expected: years. Say: "Which is why everything in today's exercise uses only what GCC 14 and Clang 18 already have."

IF BEHIND: "Read the availability column for your compiler. C++26 brings `par` to the ranges algorithms."

45. Support matrix for this session • target 1:38, ~2 min

Every row on this slide was found by building the demos on both compilers. (beat)

Row 1: everything the exercise uses works on both. Constrained algorithms, projections, the core views, `to`, `zip`, `chunk_by`, `split`, `fold_left`, `contains`.

Row 2: `views::enumerate` is missing on libc++ 18. The exercise uses `zip(iota(1uz), ...)` instead.

Rows 3 and 4: the windows, the zip family, the other folds, `find_last`, `range_adaptor_closure`. All on GCC, none on `libc++ 18`.

Row 5 goes the other way: `libc++ 18` has `starts_with`, `ends_with`, and range formatting; GCC 14 doesn't.

So neither compiler is a superset. **`libc++ 18` is the laggard on C++23 views; GCC 14 lags on range formatting and `starts_with`.** If your project builds on two toolchains, you're programming to the intersection, and this table is that intersection for today.

IF BEHIND: "The exercise uses only row 1. Neither compiler is a superset; program to the intersection."

It's 1:40. Let me give you the guidance, show you one demo, and then it's your turn.

46. Guidance for production code • target 1:40, ~2 min

This is the slide to screenshot.

Constrained algorithms and projections: now, everywhere. There's no downside.

Views in code you own, consumed once. A `filter` | `transform` | `take` inside a function body: yes. A view stored in a class member: no.

`ranges::to` at boundaries. **Return containers from functions, not views**, unless the whole point of the function is to be a view. A view in a public signature exposes a lambda type and a lifetime contract nobody will read.

Take ranges by R&& in generic code. Iterate with `auto&&`.

Measure `join`, `split`, and `reverse-of-filter` before they go in hot paths.

Test the edges when you convert a loop to a pipeline: empty input, one element, all keys equal. The `split("")` test is the example.

And `clang-tidy`'s `modernize-use-ranges`, in `clang-tidy 19` and later, converts iterator-pair calls to `ranges::` calls automatically. On a large codebase, that's the first mechanical pass. (beat)

Every bullet corresponds to something in today's exercise.

47. The same task, three ways • Demo • target 1:42, ~4 min

One task: for each sensor, print the top two readings by value, highest first. Three versions in one file, `same_task_three_ways.cpp`.

DO: Run `./build/demos/s04/demo_s04_same_task_three_ways`. Expected: the same four lines printed three times: `rpm #1 4830`, `rpm #2 4800`, `temp #1 42.5`, `temp #2 41`. Say "same output, three times."

DO: Scroll the file on the right, top to bottom. `cpp11`: spelled-out iterator types, a map of vectors, an allocation per group, an index loop with `i < rs.size() && i < 2`. `cpp20`: sort by sensor, then find each run by hand with `find_if` and a while loop,

subrange plus `take(2)`, and a manual counter. `cpp23`: sort by sensor, `chunk_by`, sort each group descending, `zip(iota(1), group | take(2))`.

The C++23 version has no index variable, no counter, and no allocation per group. And every step is a named algorithm a reviewer recognizes.

One thing to notice in `cpp23`: it sorts each group in place while `chunk_by` is iterating. That's safe here because the sort only permutes records within the group and doesn't change the sensor, so the group boundaries don't move. If the inner step changed the key, you'd corrupt the grouping. Worth a review comment if you see it.

DO: Switch to the Compiler Explorer tab with the same file, GCC 14 -02 -std=c++23. Collapse to the `cpp20` and `cpp23` functions in the assembly pane and compare their size and shape. Say: "Comparable. This isn't a benchmark, it's a sanity check that `chunk_by` didn't bring a runtime tax." Recovery: if Compiler Explorer is down, skip the assembly; the three-way source comparison is the point.

IF BEHIND: Run the binary, show only the `cpp23` function, skip Compiler Explorer. Say: "Same output, no counters, no map."

Your turn.

48. Exercise: ranges · target 1:46, ~1 min to launch (exercise runs to 1:56)

See the Exercise coaching section below for the full launch, circulation, and debrief. The short version to say:

Open `exercises/s04-ranges/README.md`. Three tasks in class. Task 1: projections. Task 2: the `top_n` pipeline. Task 3: `views::split`. **Run the tests after task 3. One will fail. That's the task.**

49. Session 4 takeaway · target 1:58, ~2 min

Ranges are the standard algorithms with the iterator pairs removed, plus lazy views that compose. Projections end comparator lambdas. Pipelines read top to bottom. `to` materializes.

The lifetime rules are the same as every other view: `string_view`, `span`, now pipelines. Nothing new to learn, just more places to apply it. And when a loop becomes a pipeline, **the tests define the contract.** (pause)

At home: tasks 4 to 8. The `const` view and the dangling `split` are the two worth doing even if you skip the rest. The `solution/` folder is next session's starter, so take a look before we meet.

Next session: concurrency with `jthread` and `stop_token`, `latch` and `barrier`; coroutines and `std::generator`, which is a range, so today plugs straight in; modules; and the adoption roadmap for your own codebase.

The support matrix is on slide 45 and in `handouts/toolchain-support-matrix.md`. Thanks, everyone.

Exercise coaching

Launch (60 seconds, at about 1:46).

DO: Switch the right pane to `exercises/s04-ranges/README.md`. Paste the command into the Teams chat: `cmake --build build && ctest --test-dir build -R s04 --output-on-failure`.

Say: "Starter is the Session 3 solution. The slide says twenty minutes; we have about ten, so here's the order. Do task 1 first, it's three one-line changes. Then skip straight to task 3, because the failing test is the thing I want you to see. Task 2 if you have time. Run the tests after each task; the `report-diff` test, `s04_report_identical`, tells you if you changed the output. Clang folks: you're on `libc++`, and the in-class tasks avoid everything it's missing. I'll call time at five minutes left."

What to watch for while they work (circulate by asking for a shared screen or a pasted error in chat):

- Task 1, `minmax`: people write `auto [lo, hi] = std::ranges::minmax(records, {}, &Record::value);` and then use `lo` as a double. It's a `Record`. Use `lo.value`.
- Task 1, the missing `{}`: `std::ranges::lower_bound(records, ts, &Record::ts)` passes the projection as the comparator. Error says "no match for call". Add `{}`.
- Task 1, `lower_bound` precondition: records must be sorted by `ts`. If someone's test fails here, ask whether the data is sorted.
- Task 1, `find_sensor`: `ranges::find` returns an iterator. If `find_sensor` returns a pointer or an optional in the starter, they must still compare against `kSensors.end()` before converting. Watch for a dereference of the end iterator.
- Task 2: `std::ranges::to<std::vector>` without the trailing `()`. The error is long and mentions templates. Point at the parentheses.
- Task 2: trying to `stable_sort` the filter view directly. Concept error about `random_access_range`. Materialize first; that's why `to` is in the pipeline.
- Task 3: the transform lambda must take `auto&&` (or `auto`) and construct `std::string_view(part)` explicitly. A lambda taking `std::string_view` by parameter won't convert implicitly from the split subrange.
- Task 3: someone will "fix" the failing test by editing the test's expected value without thinking. That's a legitimate decision only if they can say why the new contract is better and who depends on the old one. Ask them.
- Task 3 on `libc++`: no differences; `views::split` and `to` are both there.

Five minutes left (about 1:51). Say: "Five minutes. If you haven't run task 3's tests yet, do that now even if task 2 isn't done. I want everyone to see the failure."

Debrief (about 1:55, 3 minutes).

DO: Open the solution's `split` (it matches slide 21). Point at the early return.

Say: "Who saw the empty-string test fail? (pause) What did you decide?"

ASK: "Zero pieces or one: which contract, and where should the decision live?"

Expected: mixed on the contract; the answer to "where" is "in the tests". Say: "Either contract is defensible. The solution keeps the loop's contract, one empty field, with an explicit early return, so the decision is visible in code and pinned by a test. **What's not acceptable is a refactor that changes behavior and nobody notices.** Your tests caught it today. In a codebase without that test, it would have shipped."

Then: "Tasks 4 through 8 at home. Four is the `const` view. Eight is the dangling split. Do those two."

Close. Go to slide 49.

Likely questions and answers

Q: Our certified toolchain is GCC 11 (or older). How much of this can we use? A: GCC 10 first shipped `<ranges>`, but several C++20 defect reports that changed views, like `owning_view` and the reworked `views::split`, arrived around GCC 12, so on 11 some of today's slides behave differently. Constrained algorithms and projections are the safe subset on older C++20 toolchains. Everything badged C++23 needs GCC 13 or 14 at least. Check the specific feature against your toolchain rather than trusting a version number from me.

Q: Do views cost anything at runtime? A: At -O2, `filter | transform | take` compiles to the loop you'd write; the cost model slide shows it. The exceptions are `reverse over filter`, which runs the predicate twice per element, and `join/split`, which are forward-only and branchy. At -O0, every adaptor layer is a call, so debug builds of heavy pipelines are noticeably slower.

Q: Does MISRA or AUTOSAR allow ranges? A: MISRA C++:2023 is written against C++17, so C++20 ranges are simply outside its scope; it neither permits nor forbids them, and your deviation or extension process decides. AUTOSAR C++14 predates them entirely. Constrained algorithms with projections are the easiest sell, since they reduce hand-written comparators and iterator mismatches; views need a written lifetime policy.

Q: Can I put views in a public API or across a library boundary? A: Avoid it. A view's type includes lambda types and adaptor internals, so it's unnameable, ties callers to your implementation, and carries a lifetime contract that's easy to violate. Return a container (to at the boundary) or a `std::span`; take ranges as R& templates or as `std::span` for non-template interfaces.

Q: What about ABI? Is it safe to mix object files built with different compiler versions in C++20 mode? A: Views are header-only templates, so they're instantiated into your objects; the risk is passing them between separately compiled components whose library versions differ. GCC treated its C++20 library support as experimental for several releases, and layouts of some views changed with the DRs. Keep views internal and don't let them cross a component boundary.

Q: Why can't I use `std::execution::par` with `std::ranges::sort`? A: C++20 and C++23 don't have parallel overloads of the range algorithms. C++26 adds them, with the policy as the first argument. Until then, use `std::sort(par, v.begin(), v.end())`.

Q: Is it safe for two threads to iterate the same view? A: Not for views whose `begin()` caches, like `filter`, `drop_while`, `split`, and `chunk_by`: the first `begin()` call writes the cache, so two threads calling it is a data race. Share the underlying container and give each thread its own view; views are cheap to create.

Q: Will ranges hurt our compile times? A: `<ranges>` is a heavy header, and deep pipelines create long template instantiation chains, so yes, measurably in big translation units. Precompiled headers help today; `import std`; in Session 5 is the longer-term fix, toolchain permitting.

Q: Is `std::ranges::sort` faster than `std::sort`? A: Same algorithm, same codegen in practice; on libstdc++ the ranges version forwards to the same implementation. The win is correctness and readability, not speed.

Q: How do I debug a pipeline when the result is wrong? A: Break it into named stages (`auto s1 = v | filter(...)`;) and print each with a loop, or with `println("{", s1)` where range formatting exists. Keep predicates pure so that inspecting a stage doesn't change the result. In GDB, `skip -rfu ^std::` stops you stepping into library internals.

Q: `views::split` or `views::lazy_split`? A: For strings, `split`: it yields contiguous subranges you can turn into `string_views`. `lazy_split` is the original C++20 design kept for input ranges; its inner ranges aren't contiguous, so you can't make a `string_view` from them.

Q: Should `compute_stats` and `load_stream` become pipelines too (task 7)? A: Probably not. `compute_stats` mutates a map with `try_emplace` per record, which is a fold into a map, clearer as a loop. `load_stream` reads from a stream with side effects, where view lookahead would consume extra input. The criteria are consumed-once, no side effects, and more readable than the loop.

Deck issues found

Status: fixed in the deck, demos, outline and README on 2026-10-02 (see `scripts/README.md`). Items kept for the record. Still open: the Compiler Explorer <add short link> placeholders in every demo file header.

- Slide 4 notes: "Projections remove all three" lambdas. Only two are `compare-by-member` (the `stable_sort` and `lower_bound` comparators); the `copy_if` predicate captures sensor and survives as the `filter` lambda on slide 19. Projections also replace the `min/max` loop.
- Slide 11, second bullet: `ranges::begin` dispatch order is arrays, then member `begin()`, then ADL free `begin()`, not "member, free, arrays". It also rejects rvalues of non-borrowed ranges.
- Slide 12, line 2 and slide 22 notes: `load().records | views::take(3)` with `records` a data member is safe in C++20, because the rvalue vector is moved into an

owning_view (P2415) that lives for the loop (verified with a destructor print on GCC without P2718). The dangling case is an accessor returning a reference, e.g. `load().records()`. Slide 22's note that `load() | views::take(3)` is unsafe to iterate in range-for before C++23 is wrong for the same reason.

- Slide 15, `sized_range` row: "all of the above except `forward_list` and `filter`" also wrongly includes the `istream` view and `views::split` results, which are not sized.
- Slide 20: says "Fourteen adaptors"; the slide and `adaptors_tour.cpp` show twelve (plus a chain). `elements` and `common` from the outline are missing.
- Slide 25 and `cost_model.cpp` comment: the explanation for reverse-of-filter's double predicate evaluation ("reverse needs the end") doesn't apply to a filter over a vector, which is common. The cause is `std::reverse_iterator::operator*` decrementing a copy of the base iterator, so each element's backward scan runs twice (measured: 12 predicate calls vs 6 for six elements).
- Slide 31 and `chunk_by.cpp`: the "Before (C++11)" code uses structured bindings (C++17) and `std::println` (C++23).
- Slide 33 / `zip_family.cpp`: `src[0].empty()` after `as_rvalue` prints true in practice, but a moved-from `std::string` is only valid-but-unspecified. Also, the note "the pattern must be a range" is incomplete: `join_with` also accepts a single element (`join_with(' ', '')`). `as_const` is named in the outline and the demo header but never shown.
- Slides 35 and 44: `ranges::find_last` and `ranges::iota` are listed as GCC 14; `libstdc++ 13` already defines `__cpp_lib_ranges_find_last` and `__cpp_lib_ranges_iota` (verified on GCC 13.3).
- Slide 37 and `parallel.cpp` lines 27 to 29: `t1` is overwritten after the reshuffle, so the printed seq time includes the second shuffle (about 70 ms for 5M doubles on a 2-core box), inflating seq relative to par. Fix: take `t1` once before the shuffle, and a separate timestamp after it.
- Slide 43: "`std::ranges::` versions of every `<algorithm>` function" overstates it; `shift_left/shift_right` got `ranges::` versions only in C++23 (as slide 44 says), and `lexicographical_compare_three_way` has none.
- Timing, slides 46 to 49: the Exercise segment starts at 1:40 and must hold the guidance slide, the three-ways demo, a "20 minute" in-class exercise, and the takeaway in 20 minutes. Realistically about 10 minutes of hands-on time remain; the syllabus itself puts wrap-up at 1:55. Either trim a content segment by 8 to 10 minutes or relabel the exercise as 10 minutes.
- Outline vs deck: the outline promises 50 slides; the deck has 49 (outline items 33 and 34 merged into deck slide 33). Outline also lists demo files `split_edge.cpp`, `join_with.cpp`, and `searchers.cpp` that don't exist; those slides are hand-typed or use other files.
- Slide 49 notes point at `handouts/cheat-sheet-ranges.md`, which is a one-line stub.
- All demos/`s04/*`.cpp headers still read `Compiler Explorer: <add short link>`.